

EARIN Laboratory 1 - Variant 4

Searching and optimization

Group 24

By: Sai Koushik Neriyanuri (317100)

Gülten Sezer (K-7758)

Introduction

In this laboratory exercise, the task is to minimize a given function using Newton's method algorithm. Newton's method is an iterative optimization algorithm commonly used for finding the roots of a real-valued function. In this context, it is adapted to minimize a given function by iteratively updating the solution towards the minimum point.

Advantages:

- Fast convergence, especially for smooth functions.
- Efficient for high-dimensional problems, facilitating complex optimizations.
- Guaranteed convergence under specific conditions.

Disadvantages:

- Sensitive to initial guess, prone to converging to local minima.
- Computationally expensive due to Hessian matrix operations.
- Limited applicability to non-smooth functions.

Algorithm Description

Newton's method involves iteratively updating the solution based on the gradient and Hessian matrix of the function.

- **Function Definition:** The function to be minimized is defined as $f(x,y)=2\sin(x)+3\cos(y)$.
- **Gradient Calculation:** The gradient of the function with respect to x and y is computed as $\nabla f(x,y)=[2\cos(x), -3\sin(y)]$.
- **Hessian Calculation:** The Hessian matrix of the function is computed as $H(x,y)=\begin{bmatrix} -2\sin(x) & 0 \\ 0 & -3\cos(y) \end{bmatrix}$
- **Newton's Method Iteration:** The iteration updates the solution x based on the gradient and the inverse of the Hessian matrix until convergence is achieved within a specified tolerance.

Implementation

function(x, y): This function defines the objective function $f(x,y)=2\sin(x)+3\cos(y)$.

gradient(x, y): Computes the gradient of the objective function with respect to x and y .

hessian(x, y): Computes the Hessian matrix of the objective function.

newton_method(initial_guess, learning_rate=1.0, tol=1e-6, max_iter=1000): Performs Newton's method for function minimization with a learning rate. It iteratively updates the solution using the gradient, Hessian, and the specified learning rate until convergence or reaching the maximum number of iterations.

visualize(): Visualizes the objective function surface using matplotlib's 3D plotting capabilities.

Example usage: Demonstrates how to use the `newton_method` function with an initial guess and a specified learning rate, and prints the resulting minimum approximation along with the number of iterations required

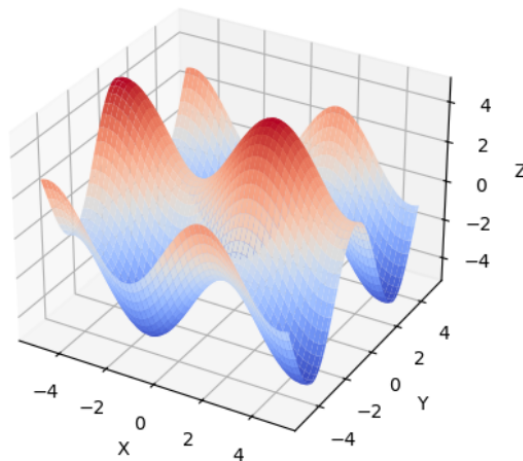
Discussion

The solution obtained using the provided code, with an initial guess of $[2.0, 2.0]$, converges to a minimum approximation of $[1.57079633, 3.14159265]$ after 6 iterations.

The result aligns with expectations, indicating the algorithm's effectiveness in minimizing the objective function. The rapid convergence within 6 iterations highlights the efficiency of Newton's method for smooth functions like the one in this scenario.

The choice of initial guess is crucial, as it influences the convergence behavior. In this case, the relatively close initial guess aids in quick convergence. However, different initial guesses may lead to varied convergence paths.

In summary, the obtained solution showcases the algorithm's capability in function minimization, while also highlighting the importance of initial guess selection in convergence behavior. Further exploration with different initial guesses can offer deeper insights into algorithm performance.



Conclusion

In conclusion, this laboratory exercise provided hands-on experience with implementing Newton's method for function minimization. Understanding the impact of initial guesses and convergence criteria is essential for effectively applying optimization algorithms. Further exploration could involve experimenting with different functions, initial guesses, and convergence parameters to gain deeper insights into optimization techniques.